

第五讲 研讨题2

一、循环

(研讨题 (3))

1. do while

C语言代码:

```
do
    body-statement;
while (text-expr);
```

go-to 版本:

```
loop:
    body-statement
    t = text-expr;
    if (t)
        goto loop;
```

2. while

C语言代码:

```
while (text-expr)
    body-statement
```

goto 版本:

(1) jump to middle

```
    goto test;
loop:
    body-statement
test:
    t = test-expr;
    if (t)
        goto loop;
```

(2) guarded-do (相当于拆出do-while循环)

```

    t = test-expr;
    if (!t)
        goto done;
loop:
    body-statement;
    while (t);
done;

```

- guarded-do优势：可以做初始测试的优化，使程序更高效

3. for

C语言：

```

for (init_expr; test_expr; update_expr)
    body-statement;

```

相当于while：

```

init_expr;
while (test_expr)
{
    body-statement;
    update-statement;
}

```

go to (jump to middle):

```

    init_expr;
    goto test;
loop:
    do
        body-statement;
        update_expr;
test:
    t = test_expr;
    if (t)
        goto loop;

```

go to (guarded do)

```

    init-expr;
    t = test-expr;
    if (!t)
        goto done;
loop:
    do
        body-statement;
        update-expr;
        while (t)
done:

```

关于popcount

研讨题 (1)

```

long pcount_goto(unsigned long x)
{
    long result = 0;    // movl $0, %eax
loop:
    result += x & 0x1;  // movq %rdi, %rdx
                        // andl $1, %edx
                        // addq %rdx, %rax
    x >>= 1;           // shrq %rdi
    if(x)
        goto loop;    // jne .L2
    return result;     // rep; ret
}

```

```

movl $0, %eax
#对低位赋值，高位会自动补零，就相当于 movl $0, %rax，但提升计算效率
.L2:
movq %rdi, %rdx
##为什么不用 movl %edi, %edx.....?
andl $1, %edx
#高位自动清零，效果相同，提升计算效率
addq %rdx, %rax
#防止溢出，坚持使用64位加法
shrq %rdi
#逐步右移x，必须移动的是完整的x
jne .L2
rep; ret

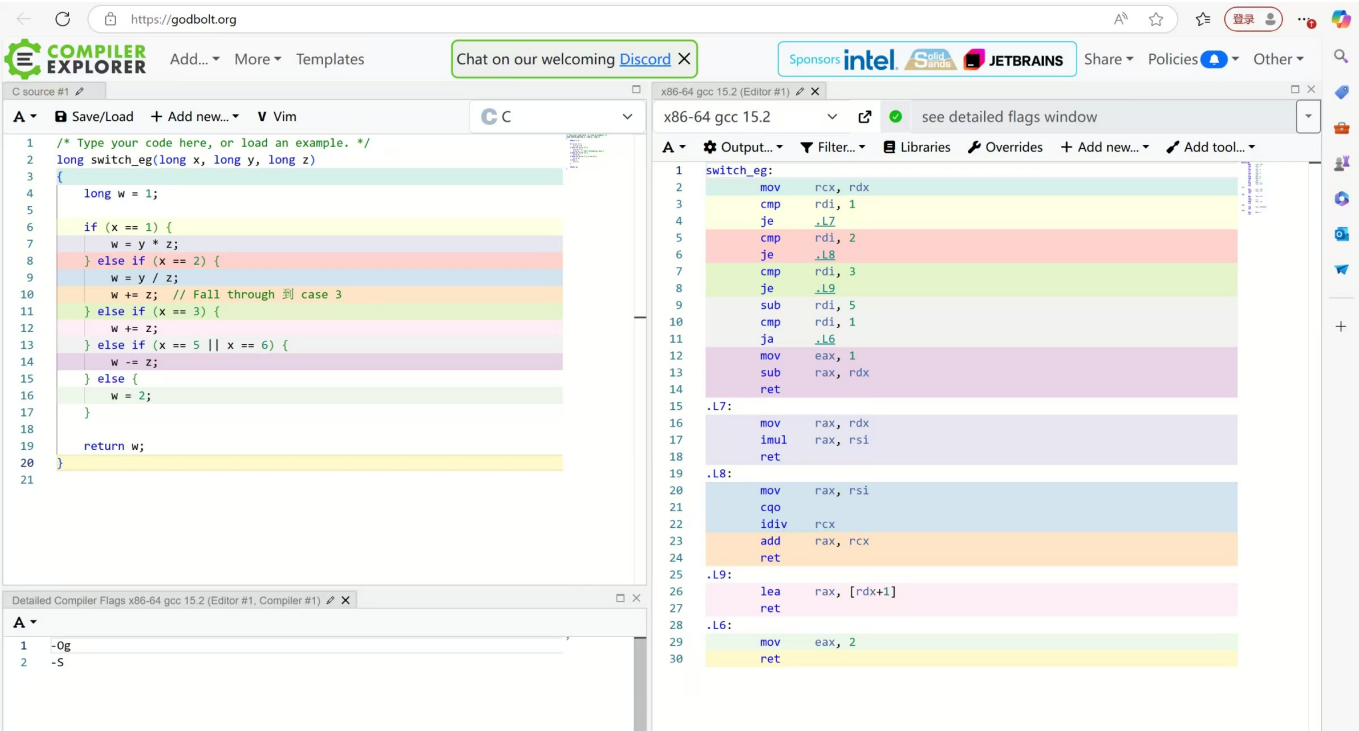
```

扩展研讨 (2)

- 指令: `popcnt %rdi, %rax`

- 编译时需加 `-mpopcnt` 参数才会使用该指令

switch---if-else改写与跳转 扩展研讨（4）



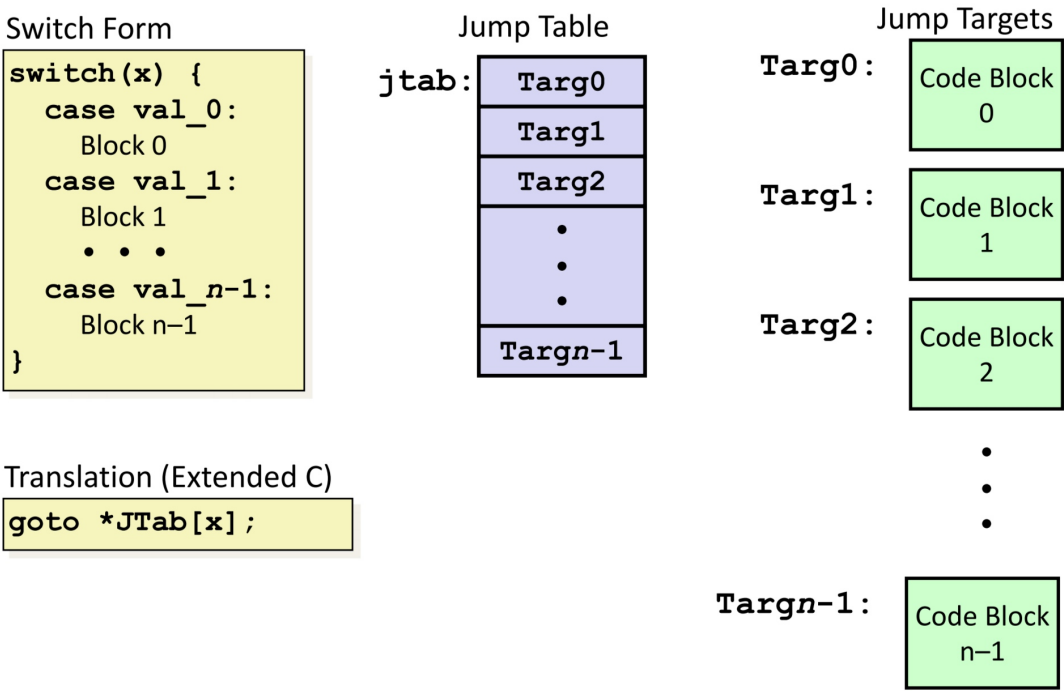
——四次跳转。

二、switch语句

1. Jump Table Structure

(研讨题（5） 基于课件第 50 页，解释图中四块内容之间的对应关系)

Jump Table Structure



50

- Switch Form: C代码结构
- 实现跳转

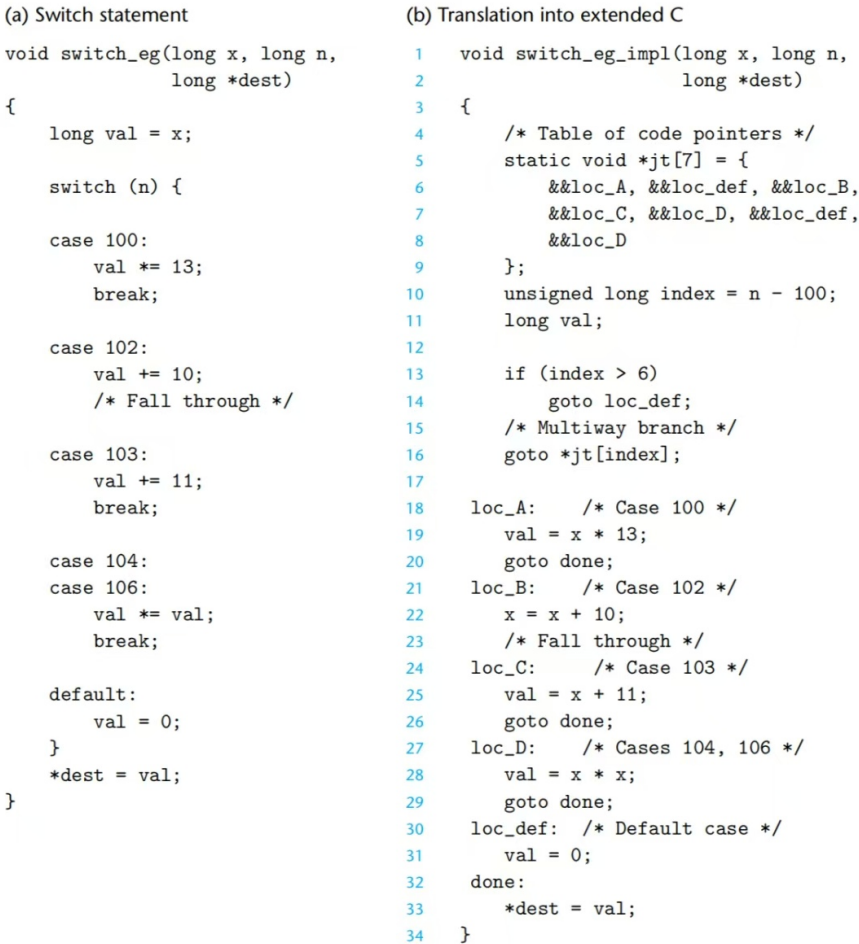


Figure 3.22 Example switch statement and its translation into extended C. The translation shows the structure of jump table jt and how it is accessed. Such tables are supported by gcc as an extension to the C language.

- Jump Table: 内存中的地址跳转表

- 跳转表中存储的是标签对应的内存地址值（64位数字）
- 标签只是汇编代码中的符号占位符,汇编器在编译过程中会将标签替换为具体的数字地址（见最后一题）
- Jump Targets: 每个case对应的代码块

2. 关于指令 `jmp *.L4(,%rdi,8)`

(研讨题 (6))

- .L4: 跳转表基地址
- %rdi: x的值
- 8: 每个地址占8字节
- 操作: 从地址 `.L4 + x*8` 处取出目标地址并跳转到目标地址对应的标签

3. 关于跳转表

(研讨题 (7))

```
.section .rodata
.align 8
.L4:
.quad .L8 # x = 0
.quad .L3 # x = 1
.quad .L5 # x = 2
.quad .L9 # x = 3
.quad .L8 # x = 4
.quad .L7 # x = 5
.quad .L7 # x = 6
```

line 1

- .section 指令用于切换当前的段 (section)
- .rodata 是段名称, 表示“只读数据”
 - 跳转表包含的是代码地址, 这些地址在程序运行时不会被修改, 因此放在只读段是安全且合理的

line 2

- 确保跳转表的起始地址是 8 字节对齐的

line 3~

- .quad 四字=八字节, 分配8个字节存储代码标签的地址
- 从上到下依次代表递增的索引对应的标签 (如注释)
 - .L4是跳转表的标签, 其它是代码块的标签
 - “.Lx”只是一个标志 (局部标签), 数字并不重要, 只要和代码中对应即可
 - 每行只对应一个局部标签, 一个局部标签却可以在不同行同时出现, 代表两个索引对应同一种情况 (e.g. case 5, case 6)
 - 哪怕对应default的索引也会被排列进来, 只是在代码里被忽视, 直接对应默认情况 (e.g. case 0, case 4)

Jump Table

Jump table

```
.section .rodata
.align 8
.L4:
.quad .L8 # x = 0
.quad .L3 # x = 1
.quad .L5 # x = 2
.quad .L9 # x = 3
.quad .L8 # x = 4
.quad .L7 # x = 5
.quad .L7 # x = 6
```

```
switch(x) {
case 1:      // .L3
    w = y*z;
    break;
case 2:      // .L5
    w = y/z;
    /* Fall Through */
case 3:      // .L9
    w += z;
    break;
case 5:
case 6:      // .L7
    w -= z;
    break;
default:    // .L8
    w = 2;
}
```

- 跳表索引总是从0开始，假如原本的x不是0，那么需要先统一调整x使它从0开始 (e.g.习题3.30)

```
void switch2(long x, long *dest)
x in %rdi
1  switch2:
2      addq    $1, %rdi
3      cmpq    $8, %rdi
4      ja      .L2
5      jmp     *.L4(,%rdi,8)
```

为跳转表生成以下代码：

```
1  .L4:
2      .quad    .L9
3      .quad    .L5
4      .quad    .L6
5      .quad    .L7
6      .quad    .L2
7      .quad    .L7
8      .quad    .L8
9      .quad    .L2
10     .quad    .L5
```

* 已知标签只是汇编代码中的符号占位符，如何查看标签对应的具体数字地址？

关于gdb表

(研讨题 (9))

Finding Jump Table in Binary

```
00000000004005e0 <switch_eg>:
4005e0:  48 89 d1                mov    %rdx,%rcx
4005e3:  48 83 ff 06             cmp    $0x6,%rdi
4005e7:  77 2b                   ja     400614 <switch_eg+0x34>
4005e9:  ff 24 fd f0 07 40 00    jmpq   *0x4007f0(,%rdi,8)
4005f0:  48 89 f0                mov    %rsi,%rax
4005f3:  48 0f af c2             imul   %rdx,%rax
4005f7:  c3                      retq
4005f8:  48 89 f0                mov    %rsi,%rax
4005fb:  48 99                   cqto
4005fd:  48 f7 f9               idiv   %rcx
400600:  eb 05                   jmp    400607 <switch_eg+0x27>
400602:  b8 01 00 00 00         mov    $0x1,%eax
400607:  48 01 c8               add    %rcx,%rax
40060a:  c3                      retq
40060b:  b8 01 00 00 00         mov    $0x1,%eax
400610:  48 29 d0               sub    %rdx,%rax
400613:  c3                      retq
400614:  b8 02 00 00 00         mov    $0x2,%eax
400619:  c3                      retq
```

Finding Jump Table in Binary (cont.)

```
00000000004005e0 <switch_eg>:
. . .
4005e9:  ff 24 fd f0 07 40 00    jmpq   *0x4007f0(,%rdi,8)
. . .
```

```
% gdb switch
(gdb) x /8xg 0x4007f0
0x4007f0:  0x0000000000400614  0x00000000004005f0
0x400800:  0x00000000004005f8  0x0000000000400602
0x400810:  0x0000000000400614  0x000000000040060b
0x400820:  0x000000000040060b  0x2c646c25203d2078
(gdb)
```

-
- 冒号前: 该行内容的起始内存位置
- 冒号后: 该行存储的两个指针值

Finding Jump Table in Binary (cont.)

```
% gdb switch
(gdb) x /8xg 0x4007f0
0x4007f0: 0x000000000000400614 0x0000000000004005f0
0x400800: 0x0000000000004005f8 0x000000000000400602
0x400810: 0x000000000000400614 0x00000000000040060b
0x400820: 0x00000000000040060b 0x2c646c25203d2078
```

```
. . . .
4005f0: 48 89 f0      mov    %rsi,%rax
4005f3: 48 0f af c2   imul   %rdx,%rax
4005f7: 58           retq
4005f8: 48 89 f0      mov    %rsi,%rax
4005fb: 48 99        cqto
4005fd: 48 f7 f9     idiv   %rcx
400600: eb 05        jmp    400607 <switch_eg+0x27>
400602: b8 01 00 00 00 mov    $0x1,%eax
400607: 48 01 c8     add    %rcx,%rax
40060a: c3          retq
40060b: b8 01 00 00 00 mov    $0x1,%eax
400610: 48 29 d0     sub    %rdx,%rax
400613: c3          retq
400614: b8 02 00 00 00 mov    $0x2,%eax
400619: c3          retq
```

gdb, 跳转表, 局部标签和索引值

- 索引值: switch(x)中的x值, C语言代码里用来指示条件判断; 经过编译器标准化处理后成为跳转表的索引
- 跳转表: 编译器在.rodata段创建的静态数组, 每个元素是8字节的代码指针, 在索引值和局部标签间建立对应关系
- 局部标签: .Lx, 是代码块的标签, 对应代码块的指针
- gdb: 跳转表的内存镜像, 查看运行时内存映射的调试工具